

TP1 Sprint Backlog

Team Project 1 — Lessons Learned System, Phase One

This backlog lists the Admin (Phase One) user stories from the *Lessons Learned System V1* user stories document that still require implementation in TP1. The New Users and baseline Admin stories (login, invitations, role assignment at invite time) are excluded, as they were already provided by the foundational code. The Admin story for adding/removing roles is also excluded, since it was already implemented in the foundational code (`guiAddRemoveRoles`).

1. Admin Sets a One-Time Password

User Story: As an admin, I can set a one-time password for a user who has forgotten their password. Using that one-time password, the user can log in to establish a new password. Once a user has logged in with the one-time password, it is cleared so it can't be used again. After setting a new password, the user must log in again.

Acceptance Criteria:

- Admin can select a user account and assign a one-time password to it.
- The one-time password satisfies the same PasswordEvaluator requirements as any other password.
- A user logging in with a valid one-time password is directed to set a permanent new password before proceeding.
- The one-time password is cleared from the account immediately after it is used.
- After the new password is set, the user must log in again with it.

Status: **Not implemented.** The Admin Home screen has a "One-Time Password" button, but its controller method (`ControllerAdminHome.setOnetimePassword`) is a stub that only prints a "Not Yet Implemented" warning.

2. Admin Deletes a User Account

User Story: As an admin, I can "delete" a user account so individuals who are no longer associated with the organization are unable to use the system. An admin is not allowed to remove that admin's own access. When deleting an account, an "Are you sure?" message must be answered with "Yes" to remove access.

Acceptance Criteria:

- Admin can select a user account and delete it.
- The system displays an "Are you sure?" confirmation before deletion proceeds.
- An admin cannot delete their own account.
- Once deleted, the account can no longer be used to log in.

Status: **Not implemented.** The "Delete a User" button exists on the Admin Home screen, but `ControllerAdminHome.deleteUser()` is a stub that only prints a "Not Yet Implemented" warning. No confirmation dialog or database deletion logic exists yet.

3. Admin Lists All User Accounts

User Story: As an admin, I can list all user accounts to help me perform my administrative functions. The list shows each account's username, the user's name, the user's email address, and the roles the user plays.

Acceptance Criteria:

- Admin can view a list of every user account in the system.

- Each row displays: username, full name, email address, and assigned role(s).
- The list reflects the current state of the database (no stale/cached data).

Status: Not implemented. `ControllerAdminHome.listUsers()` is a stub that only prints a "Not Yet Implemented" warning. No list view exists yet.

Excluded (already implemented in foundational code)

Admin Adds/Removes Roles for a User — fully implemented via `guiAddRemoveRoles.ViewAddRemoveRoles`, wired to `ControllerAdminHome.addRemoveRoles()`. Not included in this backlog per the assignment instructions to exclude stories already implemented in the provided code.